

Fair Division of Indivisible Goods

Graph-Constrained EFX Allocations

Saurabh Kumar (112402005)

Guide - Dr. Pratik Ghosal
Department of Computer Science and Engineering
Indian Institute of Technology Palakkad

May 2026

Outline

- 1 Introduction
- 2 The Settings
- 3 Graph-Constrained EFX
- 4 Sweeping Algorithm for Path Graphs
- 5 Tree-Based EFX Allocation Algorithm (Proposed)
- 6 Conclusion

Our Focus: EFX

- **EFX (Envy-Freeness up to Any Good):** Strong fairness concept
- **Definition:** No agent prefers another agent's bundle, even after removing **any single item**
- Known results:
 - EFX exists for 2 and 3 agents
- **Graph-Constrained EFX:** Fairness only required between neighboring agents

Our Contributions

① Sweeping Algorithm for Path Graphs

- Repeatedly adjusts allocations between neighboring agents
- Eliminates strong envy along the path

② Tree-Based EFX Algorithm (Proposed)

- Extends sweeping idea to tree structures
- Uses depth-first traversal with local fairness adjustments

③ Analysis

- Correctness proofs
- Python implementations

Basic Model

Definition

We consider:

- Set N of n agents
- Set M of m indivisible goods
- Agent i has additive valuation $v_i : 2^M \rightarrow \mathbb{R}_{\geq 0}$
- Allocation $A = (A_1, \dots, A_n)$ assigns disjoint bundles
- Envy graph $G = (N, E)$ specifies which agents compare

N= Set of agents (nodes)

E= Set of envy edges

Graph-EFX Condition: For every edge $(i, j) \in E$ and every $g \in A_j$:

$$v_i(A_i) \geq v_i(A_j \setminus \{g\})$$

Fairness Notions: EF

Definition (Envy-Freeness)

An allocation A is **envy-free (EF)** if every agent values their own bundle at least as much as any other agent's bundle:

$$v_i(A_i) \geq v_i(A_j) \quad \forall i, j \in N$$

Fairness Notions: EF1

Definition (Envy-Freeness up to One Good)

An allocation A is **EF1** if for every pair $i, j \in N$, there exists $g \in A_j$ such that:

$$v_i(A_i) \geq v_i(A_j \setminus \{g\})$$

Removing a single item from the envied bundle eliminates envy.

Fairness Notions: EFX

Definition (Envy-Freeness up to Any Good)

An allocation A is **EFX** if for every pair $i, j \in N$ and **every** $g \in A_j$:

$$v_i(A_i) \geq v_i(A_j \setminus \{g\})$$

Key Difference from EF1

EF1 allows a **specific** good to be removed; EFX requires envy elimination after removing **any** good.

Algorithm 1: Round-Robin

Algorithm 1 Round-Robin Algorithm

```
1: Input: Instance  $\langle N, M, v \rangle$  with  $n$  agents,  $m$  goods
2: Output: Allocation  $A = (A_1, \dots, A_n)$ 
3: for each agent  $i \in N$  do
4:    $A_i \leftarrow \emptyset$ 
5: end for
6: for  $t = 1, \dots, m$  do
7:   Let  $i \leftarrow t \bmod n$ 
8:   Let  $g^* \in \arg \max_{g \in M} v_i(g)$ 
9:    $A_i \leftarrow A_i \cup \{g^*\}$ 
10:   $M \leftarrow M \setminus \{g^*\}$ 
11: end for
```

Round-Robin: EF1 Guarantee

Statement

For additive valuations, the Round-Robin algorithm always gives an EF1 allocation.

- Agents pick items one by one in turns.
- If an agent envies another, that envy is caused by at most one item.
- Removing that item removes the envy.

So, Round-Robin always guarantees fairness up to one item (EF1).

Algorithm 2: Envy-Cycle Elimination

Algorithm 2 Envy-Cycle Elimination Algorithm

```
1: Input: Instance  $\langle N, M, v \rangle$  with  $n$  agents,  $m$  goods
2: Output: Allocation  $A = (A_1, \dots, A_n)$ 
3: for each agent  $i \in N$  do
4:    $A_i \leftarrow \emptyset$ 
5: end for
6: for  $t = 1, \dots, m$  do
7:   while there does not exist an unenvied agent do
8:     Find envy-cycle  $C = (i_1, \dots, i_d)$ 
9:     Resolve cycle:  $A_{i_j} \leftarrow A_{i_{j+1}}$  for  $j < d$ ,  $A_{i_d} \leftarrow A_{i_1}$ 
10:   end while
11:   Let  $i$  be an unenvied agent
12:    $g^* \in \arg \max_{g \in M} v_i(g)$ 
13:    $A_i \leftarrow A_i \cup \{g^*\}$ ,  $M \leftarrow M \setminus \{g^*\}$ 
14: end for
```

Envy-Cycle Elimination: EF1 Guarantee

Statement

The Envy-Cycle Elimination algorithm always produces an EF1 allocation for additive valuations.

- Goods are allocated one by one.
- At each step, choose an **unenvied agent** (an agent whom no one envies).
- Give that agent their favorite remaining good.
- If every agent is envied by someone:
 - Find a cycle of envy.
 - Rotate bundles along the cycle.
 - This removes the envy cycle.
- Repeat until all goods are allocated.

So, the algorithm guarantees EF1 efficiently.

Algorithm 3: EFX for Two Agents

Algorithm 3 EFX Allocation for Two Agents

```
1: Input: Goods  $g_1, g_2, \dots, g_m$  with values for agent 1
2: Sort goods in decreasing order of value for agent 1
3:  $X_1 \leftarrow \emptyset, X_2 \leftarrow \emptyset$ 
4: for each good  $g$  in sorted order do
5:   if  $v_1(X_1) \leq v_1(X_2)$  then
6:      $X_1 \leftarrow X_1 \cup \{g\}$ 
7:   else
8:      $X_2 \leftarrow X_2 \cup \{g\}$ 
9:   end if
10: end for
11: return  $(X_1, X_2)$ 
```

EFX allocations are guaranteed to exist for:

- **Two agents:** Using cut-and-choose method

Hidden Envy and Graph-Based Fairness

Definition (HEF-k)

An allocation X is *envy-free up to k hidden goods (HEF-k)* if there exists $S \subseteq M$ with $|S| \leq k$ such that for every $i, j \in N$:

$$v_i(X_i) \geq v_i(X_j \setminus S)$$

Definition (uHEF-k)

Uniform HEF-k: HEF-k with $|S \cap X_j| \leq 1$ for all $j \in N$.

From each agent's bundle at most one good may be hidden.

Graph-Based Hidden Envy

Definition (G-HEF-k)

For graph $G = (N, E)$, allocation X is **G-HEF-k** if $\exists S \subseteq M$ with $|S| \leq k$ such that for every edge $(i, j) \in E$:

$$v_i(X_i) \geq v_i(X_j \setminus S)$$

- Fairness only required between neighbors

Strong Envy and G-EFX

Definition (Strong Envy)

Agent i has **strong envy** for agent j if $\exists g \in X_j$ such that:

$$v_i(X_j \setminus \{g\}) > v_i(X_i)$$

Definition (G-EFX)

An allocation X is **G-EFX** if for every edge $(i, j) \in E$ and every $g \in X_j$:

$$v_i(X_i) \geq v_i(X_j \setminus \{g\})$$

Chores

Chore Division Model:

- **Chores** are tasks nobody wants — so their value is always zero or negative
- Agents prefer to **avoid** chores

Definition (Strong Envy in Chores)

Agent i strongly envies j if $\exists c \in X_j$ such that:

$$v_i(X_j) > v_i(X_i \setminus \{c\})$$

Mixed (Goods and Chores)

Item Types:

- **Goods** M^+ : Desirable items (positive value)
- **Chores** M^- : Undesirable items (negative value)

Definition (Strong Envy in Mixed Settings)

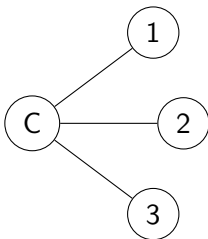
Agent i strongly envies j if either:

- $\exists g \in X_j: v_i(X_j \setminus \{g\}) > v_i(X_i)$
- $\exists c \in X_i: v_i(X_j) > v_i(X_i \setminus \{c\})$

EFX on Stars

Definition (Star Graph)

A star graph $K_{1,n-1}$ consists of one central agent connected to all others; outer agents are not connected to each other.



EFX on Stars:

Theorem (EFX on Star Graphs)

For any number of agents arranged as a star graph, there exists a G -EFX allocation under general valuations.

Idea:

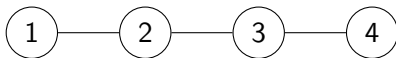
- 1 Compute EFX allocation assuming all agents have center's valuation
- 2 Outer agents pick bundles first
- 3 Remaining bundle assigned to center

This ensures no agent strongly envies another along any edge.

EFX on Path

Definition (Path Graph P_n)

Agents arranged linearly; each agent connected only to immediate neighbors.



EFX Existence on P_4

Proposition

For agents arranged in P_4 , there is always a G-EFX allocation, even with general valuations.

Idea of Proof:

- 1 Divide agents into two groups: $\{1, 2\}$ and $\{3, 4\}$
- 2 Use known results for two types of valuations
- 3 Agent 1 chooses between X_1 and X_2
- 4 Agent 4 chooses between X_3 and X_4

This ensures envy elimination along each edge.

Lexicographic Valuations

Definition (Lexicographic Valuation)

Each agent has strict priority order over items:

$$o_1 \succ o_2 \succ \cdots \succ o_m$$

Agents compare bundles based on highest-priority item they contain.

Definition (Graph Diameter)

Maximum shortest-path distance between any two agents.

$$\text{diam}(G) = \max_{u,v \in V} \text{dist}(u, v)$$

EFX Guarantee in Large Graphs

Theorem

If a graph has diameter at least 4, a G-EFX allocation exists for mixed goods and chores under lexicographic valuations, computable in polynomial time.

Idea:

- 1 Select two far-apart agents
- 2 Neighbors of one agent get goods
- 3 Neighbors of the other get chores
- 4 Remaining items distributed between distant agents

Sweeping Algorithm

Sweeping Algorithm Approach

- 1 All goods initially assigned to first agent
- 2 Two passes along path:
 - Forward sweep (left to right)
 - Reverse sweep (right to left)
- 3 At each step, apply **LocalEFX** on neighboring pair
- 4 Repeat until no strong envy remains

Local EFX for Two Agents

Algorithm 4 LocalEFX($X_1, X_{i+1}, v_1, v_{i+1}$)

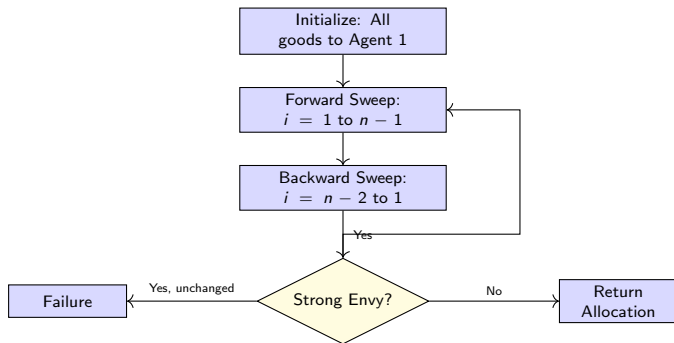
```
1:  $M' \leftarrow X_1 \cup X_{i+1}$ 
2: Sort  $M'$  in decreasing order of  $v_{i+1}$ 
3:  $Y_1 \leftarrow \emptyset, Y_2 \leftarrow \emptyset$ 
4: for each good  $g \in M'$  do
5:   if  $v_{i+1}(Y_1) \leq v_{i+1}(Y_2)$  then
6:      $Y_1 \leftarrow Y_1 \cup \{g\}$ 
7:   else
8:      $Y_2 \leftarrow Y_2 \cup \{g\}$ 
9:   end if
10: end for
11:  $X_{i+1} \leftarrow \arg \max_{Y \in \{Y_1, Y_2\}} v_{i+1}(Y)$ 
12:  $X_1 \leftarrow M' \setminus X_{i+1}$ 
13: return ( $X_1, X_{i+1}$ )
```

Algorithm 5: Sweeping Algorithm Pseudocode

Algorithm 5 Sweeping Algorithm

```
1: Input:  $n$  agents,  $m$  goods, valuations  $v_i$ , path  $P_n$ 
2: Initialize  $X_1 = M$ ,  $X_i = \emptyset$  for  $i \neq 1$ 
3: while  $\exists$  edge  $(i, i + 1)$  with strong envy do
4:    $X^{\text{init}} \leftarrow X$ 
5:   for  $i \in \{1, \dots, n - 1\}$  do
6:      $(X_i, X_{i+1}) \leftarrow \text{LocalEFX}(X_i, X_{i+1}, v_i, v_{i+1})$ 
7:   end for
8:   for  $i \in \{n - 2, \dots, 1\}$  do
9:      $(X_i, X_{i+1}) \leftarrow \text{LocalEFX}(X_i, X_{i+1}, v_i, v_{i+1})$ 
10:  end for
11:  if  $X^{\text{init}} = X$  and strong envy exists then
12:    return Failure
13:  end if
14: end while
15: return  $X$ 
```

Sweeping Algorithm Flowchart



Sweeping Algorithm: G-EFX Guarantee

Statement

If the Sweeping Algorithm stops, it always gives a G -EFX allocation on a path graph.

- The algorithm removes strong envy between neighboring agents.
- No agent strongly envies any neighbor at termination.
- This ensures fairness up to any good along the graph.

So, the final allocation satisfies G -EFX.

Tree-Based EFX Allocation

Use DFS order to extend sweeping from paths to general trees

Algorithm Overview

- 1 Assign all items to **root** agent
- 2 Compute DFS traversal order of tree
- 3 **Forward Pass** (root $\rightarrow$ leaves): Apply LocalEFX on each edge
- 4 **Backward Pass** (leaves $\rightarrow$ root): Apply LocalEFX in reverse
- 5 Repeat until no strong envy remains

Pre-order

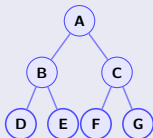
A **pre-order** traversal is a way to visit every node in a tree (a hierarchical graph with no cycles). The rule for pre-order is:

- 1 Visit the current node
- 2 Traverse the left subtree
- 3 Traverse the right subtree

Easy way to remember:

Root → Left → Right

Example Tree



Adjacency List

Node	
A	[B, C]
B	[A, D, E]
C	[A, F, G]
D	[B]
E	[B]
F	[C]
G	[C]

Pre-order Output

Visit order:

A → B → D
→ E → C → F
→ G

A B D E C F G

Pre-order Edge List

Edges are recorded during DFS when first traversing from parent to child.

Pre-order edges=[(A,B), (B,D), (B,E), (A,C), (C,F), (C,G)]

Post-order

A **post-order** traversal is a way to visit every node in a tree (a hierarchical graph with no cycles).

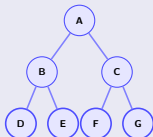
The rule for post-order is:

- 1 Traverse the left subtree
- 2 Traverse the right subtree
- 3 Visit the current node

Easy way to remember:

Left → Right → Root

Example Tree



Adjacency List

Node	
A	[B, C]
B	[A, D, E]
C	[A, F, G]
D	[B]
E	[B]
F	[C]
G	[C]

Post-order Output

Visit order:

D → E → B
→ F → G → C
→ A

D E B F G C A

Post-order Edge List

An edge is recorded when returning from a child node back to its parent after fully exploring the subtree.

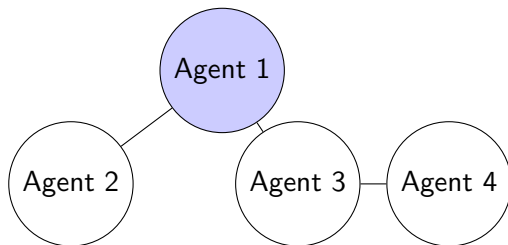
Post-order edges=[(D,B), (E,B), (B,A), (F,C), (G,C), (C,A)]

Algorithm 6: Tree-Based EFX Allocation

Algorithm 6 Tree-Based EFX Allocation

```
1: Initialize:  $X_r \leftarrow M$ ,  $X_i \leftarrow \emptyset$  for  $i \neq r$ 
2: Compute DFS order:  $[e_1, e_2, \dots, e_{n-1}]$  from root  $r$ 
3: while  $\exists$  edge  $(i, j) \in E$  with strong envy do
4:    $X^{\text{init}} \leftarrow X$ 
5:   for each edge (parent, child) in DFS pre-order do
6:      $(X_{\text{parent}}, X_{\text{child}}) \leftarrow \text{LocalEFX}(X_{\text{parent}}, X_{\text{child}}, v_{\text{parent}}, v_{\text{child}})$ 
7:   end for
8:   for each edge (child, parent) in DFS post-order do
9:      $(X_{\text{child}}, X_{\text{parent}}) \leftarrow \text{LocalEFX}(X_{\text{child}}, X_{\text{parent}}, v_{\text{child}}, v_{\text{parent}})$ 
10:  end for
11:  if  $X^{\text{init}} = X$  and strong envy exists then
12:    return Failure
13:  end if
14: end while
15: return  $X$ 
```

Tree Structure Example



4 agents arranged in a tree with root Agent 1.

Tree Example: Step-by-Step

Step 1: Initial Allocation

$$X_1 = \{g_1, g_2, g_3, g_4\}, \quad X_2 = X_3 = X_4 = \emptyset$$

Step 2: Forward Sweep (root $\rightarrow$ leaves)

- Edge (1,2): LocalEFX divides goods between Agents 1 and 2
- Edge (1,3): Remaining goods split between Agents 1 and 3
- Edge (3,4): Allocation adjusted between Agents 3 and 4

Step 3: Backward Sweep (leaves $\rightarrow$ root)

- Edge (2,1) $\rightarrow$ Edge (4,3) $\rightarrow$ Edge (3,1)

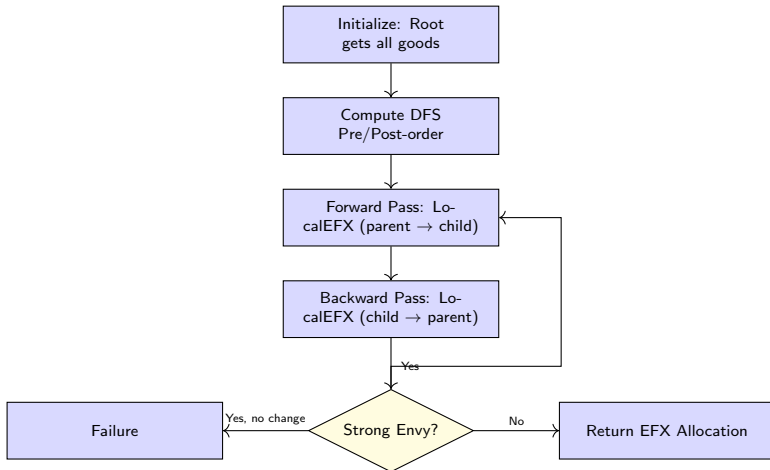
Tree Example: Final Allocation

Final Allocation (Example Outcome):

$$X_1 = \{g_1\}, \quad X_2 = \{g_2\}, \quad X_3 = \{g_3\}, \quad X_4 = \{g_4\}$$

Although agents have different preferences, the algorithm successfully reduces strong envy step by step.

Tree-Based Algorithm Flowchart



Tree-Based Algorithm: G-EFX Guarantee

Statement

If the Tree-Based Algorithm stops successfully, it always produces a G-EFX allocation on a tree graph.

- The algorithm reduces strong envy between neighboring agents.
- Each connected pair of agents becomes EFX fair.
- This guaranties fairness across the entire tree.

So, the final allocation satisfies G-EFX.

Summary of Contributions

Main Contributions:

- **Sweeping Algorithm** for path graphs achieving G -EFX
- **Tree-Based EFX Algorithm** extending the approach to trees

Thank You!

Questions?

Saurabh Kumar (112402005)

IIT Palakkad

May 2026